

APPROFONDISSEMENTS EN BASE DE DONNÉES

MySQL

Responsable cours :	Pr. Samba DIAW
Responsable TP :	Amadou MBAYE
Filière :	L2 GLSI
SGBD :	MySQL
Langage :	Au choix de l'étudiant
Type d'application :	Au choix de l'étudiant
Date de rendu :	6 juin 2026 à 23h59
Travail :	Groupe de 3 ou 4 étudiants

Table des matières

Consignes générales	3
1 Sujet 1 : Gestion d'une Bibliothèque Universitaire	5
1.1 Contexte	5
1.2 Entités et règles de gestion	5
1.3 Partie 1 : Modélisation	5
1.4 Partie 2 : Base de données MySQL	5
1.5 Partie 3 : Gestion des droits (LCD)	6
1.6 Partie 4 : Application cliente	6
2 Sujet 2 : Gestion d'une Clinique Vétérinaire	7
2.1 Contexte	7
2.2 Entités et règles de gestion	7
2.3 Partie 1 : Modélisation	7
2.4 Partie 2 : Base de données MySQL	7
2.5 Partie 3 : Gestion des droits (LCD)	8
2.6 Partie 4 : Application cliente	8
3 Sujet 3 : Gestion d'une Agence Immobilière	9
3.1 Contexte	9
3.2 Entités et règles de gestion	9
3.3 Partie 1 : Modélisation	9
3.4 Partie 2 : Base de données MySQL	9
3.5 Partie 3 : Gestion des droits (LCD)	10
3.6 Partie 4 : Application cliente	10
4 Sujet 4 : Gestion d'un Restaurant	11
4.1 Contexte	11
4.2 Entités et règles de gestion	11
4.3 Partie 1 : Modélisation	11
4.4 Partie 2 : Base de données MySQL	11
4.5 Partie 3 : Gestion des droits (LCD)	12
4.6 Partie 4 : Application cliente	12
5 Sujet 5 : Gestion d'une École de Conduite	13
5.1 Contexte	13
5.2 Entités et règles de gestion	13
5.3 Partie 1 : Modélisation	13
5.4 Partie 2 : Base de données MySQL	13
5.5 Partie 3 : Gestion des droits (LCD)	14
5.6 Partie 4 : Application cliente	14
6 Sujet 6 : Gestion d'une Agence de Voyage	15
6.1 Contexte	15
6.2 Entités et règles de gestion	15
6.3 Partie 1 : Modélisation	15
6.4 Partie 2 : Base de données MySQL	15

6.5	Partie 3 : Gestion des droits (LCD)	16
6.6	Partie 4 : Application cliente	16
7	Sujet 7 : Gestion d'un Entrepôt et de ses Stocks	17
7.1	Contexte	17
7.2	Entités et règles de gestion	17
7.3	Partie 1 : Modélisation	17
7.4	Partie 2 : Base de données MySQL	17
7.5	Partie 3 : Gestion des droits (LCD)	18
7.6	Partie 4 : Application cliente	18
8	Sujet 8 : Gestion d'un Club Sportif	19
8.1	Contexte	19
8.2	Entités et règles de gestion	19
8.3	Partie 1 : Modélisation	19
8.4	Partie 2 : Base de données MySQL	19
8.5	Partie 3 : Gestion des droits (LCD)	20
8.6	Partie 4 : Application cliente	20
9	Sujet 9 : Gestion d'une Résidence Universitaire	21
9.1	Contexte	21
9.2	Entités et règles de gestion	21
9.3	Partie 1 : Modélisation	21
9.4	Partie 2 : Base de données MySQL	21
9.5	Partie 3 : Gestion des droits (LCD)	22
9.6	Partie 4 : Application cliente	22
10	Sujet 10 : Gestion d'une Entreprise et de sa Paie	23
10.1	Contexte	23
10.2	Entités et règles de gestion	23
10.3	Partie 1 : Modélisation	23
10.4	Partie 2 : Base de données MySQL	23
10.5	Partie 3 : Gestion des droits (LCD)	24
10.6	Partie 4 : Application cliente	24

Consignes Générales

Objectifs pédagogiques

À l'issue de ce mini-projet, vous serez capable de :

- Modéliser un système d'information (MCD / MLD)
- Créer et administrer une base de données MySQL (LDD)
- Insérer, modifier et interroger des données (LMD)
- Gérer les droits des utilisateurs (LCD)
- Connecter une application cliente à MySQL
- Implémenter les opérations CRUD dans un langage de programmation

Attribution des sujets

Un sujet est attribué par groupe. Deux groupes ayant rendu un rapport identique ou fortement similaire se verront appliquer une **pénalité sur la note finale**. Chaque groupe est responsable de l'originalité de son travail.

Travail demandé pour chaque sujet

Chaque mini-projet est structuré en quatre parties identiques :

1. **Partie 1 : Modélisation** : réaliser le MCD avec Merise et en déduire le MLD complet avec clés primaires (soulignées) et clés étrangères (#).
2. **Partie 2 : Base de données MySQL** :
 - Créer la base et toutes les tables avec les contraintes appropriées (PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK, DEFAULT, ON DELETE CASCADE)
 - Insérer des données de test réalistes (minimum indiqué dans chaque sujet)
 - Écrire les requêtes de consultation demandées (jointures, GROUP BY, sous-requêtes, fonctions d'agrégation)
3. **Partie 3 : Gestion des droits (LCD)** : créer les utilisateurs MySQL indiqués et leur attribuer les privilèges appropriés avec GRANT et REVOKE.
4. **Partie 4 : Application cliente** : développer une application (langage et type au choix) qui se connecte à MySQL et implémente les fonctionnalités demandées. Les **requêtes préparées** (*prepared statements*) sont obligatoires.

Exigences pour le rapport

Le rapport doit être **clair, structuré et détaillé**. Un rapport bâclé, incomplet ou sans explications sera pénalisé. Il doit obligatoirement contenir :

- Une page de garde : noms, prénoms, classe, intitulé du sujet
- Une introduction présentant le contexte et les objectifs du projet
- Le MCD (schéma dessiné ou généré avec un outil)
- Le MLD textuel complet avec toutes les clés indiquées
- La justification des choix techniques : langage utilisé, type d'application, outils utilisés

- Une description de chaque fonctionnalité implémentée avec captures d'écran à l'appui
 - Les requêtes SQL importantes commentées et expliquées
 - Une conclusion mentionnant les difficultés rencontrées et les améliorations possibles
- Longueur minimale attendue : 10 pages.

Livrable

Rapport complet(fichier pdf) selon les exigences ci-dessus.

Soumission

Le livrable est à déposer **exclusivement** via le lien suivant avant le **6 juin 2026 à 23h59** via le lien ci-dessous :

Rapports L2 GLSI-ESP — MBAYE Amadou

<https://mbayeamadou.com/soumission-rapports-projet-base-de-donnees/>

Important : Tout travail non personnel est sanctionné par une pénalité sur la note. Deux rapports identiques ou similaires entraîneront une pénalité sur la note des deux groupes concernés. Une soutenance en ligne sera organisée à l'issue du rendu. Aucun dépôt hors délai ne sera accepté.

1. Sujet 1 : Gestion d'une Bibliothèque Universitaire

1.1. Contexte

La bibliothèque du département Génie Informatique souhaite remplacer son registre papier par une application informatique. Elle gère des livres, des adhérents (étudiants et enseignants), des exemplaires physiques et des emprunts. Une amende de **100 FCFA par jour de retard** est appliquée automatiquement.

1.2. Entités et règles de gestion

- **Livre** : ISBN, titre, auteur, catégorie, nombre d'exemplaires
- **Adherent** : numéro, nom, prénom, filière (Génie Logiciel/Telecom), niveau, email, date d'inscription
- **Exemplaire** : identifiant, livre associé, état (**disponible** / **emprunté** / **abîmé**)
- **Emprunt** : adhérent, exemplaire, date d'emprunt, date de retour prévue (J+14), date de retour réelle
- **Amende** : emprunt concerné, montant, statut payé/non payé

Règles de gestion à implémenter dans l'application :

- Un adhérent ne peut pas avoir plus de **3 emprunts actifs** simultanément
- Un exemplaire **abîmé** ne peut pas être emprunté
- Un adhérent avec des amendes non payées ne peut plus emprunter
- L'amende est calculée automatiquement au retour si `dateRetourReelle > dateRetourPrevue`

1.3. Partie 1 : Modélisation

Réaliser le MCD selon la méthode Merise puis déduire le MLD complet.

1.4. Partie 2 : Base de données MySQL

Création (LDD)

Créer la base `biblio_dept` et toutes ses tables avec les contraintes appropriées.

Alimentation (LMD)

Insérer au minimum : 10 livres, 15 adhérents, 20 exemplaires, 15 emprunts dont 5 avec retard.

Requêtes de consultation

1. Liste des exemplaires disponibles avec titre et auteur
2. Nombre d'emprunts actifs par adhérent
3. Adhérents ayant des amendes non payées (sous-requête)
4. Les 3 livres les plus empruntés
5. Adhérents n'ayant jamais emprunté
6. Montant total des amendes par filière

1.5. Partie 3 : Gestion des droits (LCD)

Créer deux utilisateurs :

- `bibliothecaire` : `SELECT`, `INSERT`, `UPDATE` sur toutes les tables, pas de `DELETE`
- `lecteur` : `SELECT` uniquement sur `Livre` et `Exemplaire`

1.6. Partie 4 : Application cliente

Implémenter les fonctionnalités suivantes :

1. **Gestion des adhérents** : ajouter, modifier, supprimer, rechercher par nom ou filière
2. **Gestion des livres** : ajouter un livre, ajouter un exemplaire, lister les disponibles
3. **Enregistrer un emprunt** : vérifier les règles de gestion avant d'enregistrer
4. **Enregistrer un retour** : mettre à jour l'état de l'exemplaire et calculer l'amende
5. **Tableau de bord** : emprunts actifs, exemplaires disponibles, amendes non payées
6. **Rapport des retards** : liste des emprunts en retard avec nombre de jours de retard

Bonne chance !

2. Sujet 2 : Gestion d'une Clinique Vétérinaire

2.1. Contexte

Une clinique vétérinaire souhaite informatiser la gestion de ses patients (animaux), de leurs propriétaires, des consultations, des ordonnances et des paiements. Le vétérinaire doit pouvoir retrouver rapidement l'historique médical de chaque animal.

2.2. Entités et règles de gestion

- **Propriétaire** : numéro, nom, prénom, téléphone, adresse
- **Animal** : numéro, nom, espèce, race, date de naissance, propriétaire associé
- **Veterinaire** : numéro, nom, prénom, spécialité
- **Consultation** : animal, vétérinaire, date, diagnostic, montant
- **Médicament** : référence, nom, dosage, prix unitaire, stock
- **Ordonnance** : consultation associée, médicaments prescrits, posologie, durée
- **Paiement** : consultation associée, montant, date, mode de paiement

Règles de gestion :

- Un médicament ne peut pas être prescrit si son stock est à 0
- Chaque prescription décrémente automatiquement le stock du médicament
- Un animal ne peut avoir qu'une seule consultation active par jour

2.3. Partie 1 : Modélisation

Réaliser le MCD selon Merise puis déduire le MLD complet.

2.4. Partie 2 : Base de données MySQL

Création (LDD)

Créer la base `clinique_vet` et toutes ses tables avec les contraintes appropriées.

Alimentation (LMD)

Insérer au minimum : 3 vétérinaires, 12 propriétaires, 20 animaux, 10 médicaments, 25 consultations, 15 ordonnances.

Requêtes de consultation

1. Historique médical complet d'un animal donné (toutes consultations + diagnostics)
2. Médicaments en rupture ou sous le seuil de 5 unités
3. Propriétaires ayant plus de 2 animaux enregistrés
4. Chiffre d'affaires mensuel de la clinique (GROUP BY mois)
5. Vétérinaire ayant effectué le plus de consultations
6. Animaux n'ayant jamais eu de consultation

2.5. Partie 3 : Gestion des droits (LCD)

Créer deux utilisateurs :

- **veterinaire** : tous les droits sauf suppression sur **Animal** et **Proprietaire**
- **secretaire** : SELECT, INSERT sur **Proprietaire**, **Animal**, **Paielement** uniquement

2.6. Partie 4 : Application cliente

Implémenter les fonctionnalités suivantes :

1. **Gestion des propriétaires et animaux** : ajouter, modifier, rechercher un animal par nom ou espèce
2. **Enregistrer une consultation** : vérifier qu'aucune consultation n'existe ce jour pour cet animal
3. **Gérer une ordonnance** : ajouter des médicaments à une consultation, vérifier le stock
4. **Enregistrer un paiement** : saisir le mode de paiement et marquer la consultation comme payée
5. **Fiche animal** : afficher l'historique complet d'un animal (consultations, diagnostics, ordonnances)
6. **Alerte stock** : afficher tous les médicaments dont le stock est inférieur à un seuil paramétrable

Bonne chance !

3. Sujet 3 : Gestion d'une Agence Immobilière

3.1. Contexte

Une agence immobilière gère un portefeuille de biens (appartements, villas, studios) à louer ou à vendre. Elle veut informatiser la gestion de ses biens, de ses clients, des visites, des contrats de location et des paiements de loyer.

3.2. Entités et règles de gestion

- **Bien** : référence, type (appartement/villa/studio), adresse, superficie, prix (loyer ou vente), statut (**disponible** / **loué** / **vendu**)
- **Proprietaire** : numero, nom, prenom, telephone, email
- **Client** : numero, nom, prenom, telephone, type (locataire/acheteur)
- **Agent** : numero, nom, prenom, commission en pourcentage
- **Visite** : bien, client, agent, date de visite, observation
- **Contrat** : bien, client, agent, date debut, date fin, loyer mensuel, caution
- **PaiementLoyer** : contrat, mois, annee, montant, date de paiement

Règles de gestion :

- Un bien loué ou **vendu** ne peut pas faire l'objet d'un nouveau contrat
- La caution est égale à 2 mois de loyer
- Un client ne peut avoir qu'un seul contrat actif à la fois
- Un loyer impayé depuis plus de 30 jours génère une alerte

3.3. Partie 1 : Modélisation

Réaliser le MCD selon Merise puis déduire le MLD complet.

3.4. Partie 2 : Base de données MySQL

Création (LDD)

Créer la base `agence_immo` et toutes ses tables avec les contraintes appropriées.

Alimentation (LMD)

Insérer au minimum : 3 agents, 10 propriétaires, 20 biens, 15 clients, 20 visites, 8 contrats actifs, 30 paiements.

Requêtes de consultation

1. Liste de tous les biens disponibles avec leur type et prix
2. Biens ayant eu le plus de visites sans aboutir à un contrat
3. Revenus mensuels totaux de l'agence (loyers encaissés)
4. Clients ayant des loyers impayés (sous-requête)
5. Commission totale de chaque agent sur les contrats signés
6. Biens loués depuis plus de 12 mois

3.5. Partie 3 : Gestion des droits (LCD)

Créer deux utilisateurs :

- **agent_immo** : SELECT, INSERT sur toutes les tables ; UPDATE uniquement sur Visite et Contrat
- **comptable** : SELECT sur toutes les tables ; INSERT, UPDATE sur PaiementLoyer uniquement

3.6. Partie 4 : Application cliente

Implémenter les fonctionnalités suivantes :

1. **Gestion des biens** : ajouter, modifier le statut, rechercher par type, prix ou superficie
2. **Gestion des clients** : ajouter un client, afficher son historique de visites et contrats
3. **Enregistrer une visite** : saisir un bien, un client et un agent ; vérifier que le bien est disponible
4. **Créer un contrat** : vérifier les règles, calculer automatiquement la caution, mettre le bien en statut loué
5. **Enregistrer un paiement de loyer** : saisir le mois/année, le montant ; alerter si un mois est manquant
6. **Tableau de bord** : nombre de biens disponibles, loyers du mois encaissés, alertes loyers impayés

Bonne chance !

4. Sujet 4 : Gestion d'un Restaurant

4.1. Contexte

Un restaurant souhaite informatiser la gestion de ses tables, de ses commandes, de sa carte (plats et boissons), de ses serveurs et de sa facturation. Le gérant veut également suivre les stocks d'ingrédients pour anticiper les ruptures.

4.2. Entités et règles de gestion

- **Table** : numero, capacite, statut (**libre** / **occupee** / **reservee**)
- **Serveur** : numero, nom, prenom, poste (serveur/chef de rang)
- **Categorie** : libelle (Entree, Plat, Dessert, Boisson)
- **Plat** : reference, nom, description, prix, categorie, disponible (oui/non)
- **Commande** : numero de table, serveur, date et heure, statut (**en cours** / **servie** / **payee**)
- **LigneCommande** : commande, plat, quantite, prix unitaire au moment de la commande
- **Ingredient** : reference, nom, unite, stock actuel, seuil d'alerte
- **CompositionPlat** : plat, ingredient, quantite necessaire

Règles de gestion :

- Une table **occupée** ne peut pas recevoir une nouvelle commande tant que la précédente n'est pas payée
- Un plat marqué **non disponible** ne peut pas être commandé
- La commande d'un plat décrémente les stocks des ingrédients qui le composent
- Une alerte est levée si le stock d'un ingrédient passe sous le seuil d'alerte

4.3. Partie 1 : Modélisation

Réaliser le MCD selon Merise puis déduire le MLD complet.

4.4. Partie 2 : Base de données MySQL

Création (LDD)

Créer la base `restaurant_bd` et toutes ses tables avec les contraintes appropriées.

Alimentation (LMD)

Insérer au minimum : 4 serveurs, 10 tables, 20 plats répartis en 4 catégories, 15 ingrédients, 30 commandes, 60 lignes de commande.

Requêtes de consultation

1. Chiffre d'affaires par jour sur les 7 derniers jours
2. Plats les plus commandés (TOP 5 par quantité)
3. Tables ayant une commande non payée depuis plus de 2 heures

4. Ingrédients en dessous du seuil d'alerte
5. Chiffre d'affaires par serveur (GROUP BY)
6. Plats jamais commandés depuis l'ouverture

4.5. Partie 3 : Gestion des droits (LCD)

Créer deux utilisateurs :

- `serveur_role` : SELECT sur `Plat`, `Table`; INSERT, UPDATE sur `Commande`, `LigneCommande`
- `gerant` : tous les privilèges sur toutes les tables de la base

4.6. Partie 4 : Application cliente

Implémenter les fonctionnalités suivantes :

1. **Plan de salle** : afficher le statut de chaque table (libre/occupée/réservée) en temps réel
2. **Prise de commande** : sélectionner une table et un serveur, ajouter des plats à la commande en vérifiant leur disponibilité
3. **Modifier une commande** : ajouter ou retirer un plat d'une commande en cours
4. **Générer la facture** : calculer le total d'une commande et marquer la table comme libre après paiement
5. **Gestion de la carte** : ajouter/modifier un plat, activer ou désactiver sa disponibilité
6. **Tableau de bord** : CA du jour, plats les plus commandés, alertes stock ingrédients

Bonne chance !

5. Sujet 5 : Gestion d'une École de Conduite

5.1. Contexte

Une auto-école souhaite informatiser la gestion de ses candidats, de ses moniteurs, des leçons de conduite, des examens (code et conduite) et des paiements. L'objectif est de suivre la progression de chaque candidat jusqu'à l'obtention du permis.

5.2. Entités et règles de gestion

- **Candidat** : numero, nom, prenom, date de naissance, telephone, email, date d'inscription
- **Moniteur** : numero, nom, prenom, numéro de brevet, disponibilite
- **Vehicule** : immatriculation, marque, modele, annee, boîte (manuelle/automatique), statut
- **Lecon** : candidat, moniteur, vehicule, date, heure de debut, duree en minutes, observations
- **ExamenCode** : candidat, date, résultat (reçu/recalé), score sur 40
- **ExamenConduite** : candidat, moniteur, date, resultat (reçu/recalé), observations
- **Paiement** : candidat, montant, date, motif (inscription/leçon/examen)

Règles de gestion :

- Un candidat ne peut passer l'examen de conduite que s'il a obtenu le code (score $\geq 35/40$)
- Un moniteur ne peut pas avoir deux leçons au même créneau horaire
- Un véhicule ne peut pas être affecté à deux leçons simultanées
- Un candidat ayant échoué 3 fois à l'examen de conduite est signalé

5.3. Partie 1 : Modélisation

Réaliser le MCD selon Merise puis déduire le MLD complet.

5.4. Partie 2 : Base de données MySQL

Création (LDD)

Créer la base `autoecole_bd` et toutes ses tables avec les contraintes appropriées.

Alimentation (LMD)

Insérer au minimum : 5 moniteurs, 5 véhicules, 20 candidats, 50 leçons, 15 examens code, 10 examens conduite, 30 paiements.

Requêtes de consultation

1. Candidats ayant obtenu le code et éligibles à l'examen de conduite
2. Nombre d'heures de leçons par candidat (GROUP BY + SUM)
3. Moniteur ayant le plus d'heures de leçons dispensées

4. Taux de réussite à l'examen de conduite (pourcentage)
5. Candidats ayant échoué 3 fois ou plus à l'examen de conduite
6. Revenus encaissés par type de paiement

5.5. Partie 3 : Gestion des droits (LCD)

Créer deux utilisateurs :

- `moniteur_role` : SELECT sur toutes les tables ; INSERT sur `Lecon` uniquement
- `secretariat` : SELECT, INSERT, UPDATE sur `Candidat`, `Paielement`, `ExamenCode`, `ExamenConduite`

5.6. Partie 4 : Application cliente

Implémenter les fonctionnalités suivantes :

1. **Gestion des candidats** : inscrire un candidat, consulter sa fiche de progression (leçons, examens, solde)
2. **Planifier une leçon** : vérifier la disponibilité du moniteur et du véhicule avant d'enregistrer
3. **Enregistrer un examen** : vérifier l'éligibilité du candidat (code obtenu pour conduite), saisir le résultat
4. **Gestion des paiements** : enregistrer un paiement, afficher le solde restant dû par candidat
5. **Planning moniteur** : afficher le planning hebdomadaire d'un moniteur sélectionné
6. **Tableau de bord** : candidats en attente d'examen, taux de réussite du mois, alertes (3 échecs)

Bonne chance !

6. Sujet 6 : Gestion d'une Agence de Voyage

6.1. Contexte

Une agence de voyage spécialisée dans les circuits en Afrique de l'Ouest souhaite informatiser la gestion de ses destinations, de ses circuits, de ses guides, de ses clients et de leurs réservations. L'agence propose des circuits à dates fixes et doit gérer les places disponibles.

6.2. Entités et règles de gestion

- **Destination** : identifiant, pays, ville, description, duree du visa (jours)
- **Circuit** : code, libelle, destination, duree (jours), prix par personne, nb places max
- **Depart** : circuit, date de depart, nb places restantes, statut (**ouvert** / **complet** / **annule**)
- **Guide** : numero, nom, prenom, langues parlees, telephone
- **Client** : numero, nom, prenom, nationalite, email, telephone
- **Reservation** : client, depart, guide attitre, nb personnes, statut, date de reservation
- **Paiement** : reservation, montant, date, mode (**especes** / **virement** / **carte**)

Règles de gestion :

- Le nombre de personnes d'une réservation ne peut pas dépasser les places restantes du départ
- Toute réservation confirmée décrémente automatiquement les places restantes du départ
- Un départ est automatiquement marqué **complet** quand les places restantes atteignent 0
- Un acompte de 30% du montant total est exigé à la réservation

6.3. Partie 1 : Modélisation

Réaliser le MCD selon Merise puis déduire le MLD complet.

6.4. Partie 2 : Base de données MySQL

Création (LDD)

Créer la base `agence_voyage` et toutes ses tables avec les contraintes appropriées.

Alimentation (LMD)

Insérer au minimum : 5 destinations, 8 circuits, 12 départs, 4 guides, 20 clients, 25 réservations, 20 paiements.

Requêtes de consultation

1. Départs disponibles (statut ouvert) avec places restantes
2. Chiffre d'affaires par destination (jointures multiples + GROUP BY)
3. Clients ayant réservé plusieurs circuits différents
4. Réservations ayant un solde impayé (montant total - somme des paiements > 0)
5. Guide ayant accompagné le plus de voyageurs
6. Circuits jamais réservés (NOT IN ou NOT EXISTS)

6.5. Partie 3 : Gestion des droits (LCD)

Créer deux utilisateurs :

- **agent** : SELECT, INSERT, UPDATE sur toutes les tables ; pas de DROP ni DELETE
- **comptable** : SELECT sur toutes les tables ; INSERT, UPDATE sur **Paie**ment uniquement

6.6. Partie 4 : Application cliente

Implémenter les fonctionnalités suivantes :

1. **Catalogue des circuits** : afficher les circuits avec leurs départs disponibles et places restantes
2. **Créer une réservation** : sélectionner un départ, vérifier les places, calculer l'acompte et enregistrer
3. **Gérer les paiements** : enregistrer un paiement, afficher le solde restant dû pour chaque réservation
4. **Gestion des clients** : ajouter un client, afficher son historique de réservations et de paiements
5. **Annuler une réservation** : annuler et restituer les places au départ correspondant
6. **Tableau de bord** : départs du mois, taux de remplissage des circuits, réservations avec solde impayé

Bonne chance !

7. Sujet 7 : Gestion d'un Entrepôt et de ses Stocks

7.1. Contexte

Une entreprise de distribution possède un entrepôt central qui approvisionne plusieurs points de vente. Elle souhaite numériser la gestion de ses produits, de ses fournisseurs, des entrées/sorties de stock et des commandes fournisseurs, afin d'éviter les ruptures et les surcoûts de stockage.

7.2. Entités et règles de gestion

- **Categorie** : identifiant, libelle (Alimentaire, Electronique, Textile, etc.)
- **Produit** : reference, designation, categorie, prix d'achat, prix de vente, stock actuel, stock minimum, stock maximum
- **Fournisseur** : numero, nom, contact, ville, delai de livraison moyen (jours)
- **CommandeFourn** : numero, fournisseur, date de commande, date de livraison prevue, statut
- **LigneCommande** : commande, produit, quantite commandee, prix d'achat unitaire
- **PointDeVente** : code, nom, adresse, responsable
- **MouvementStock** : produit, type (**entrée** / **sortie**), quantite, date, point de vente ou commande source, motif

Règles de gestion :

- Tout mouvement de stock **sortie** vérifie que `stockActuel` $\geq$ quantité demandée
- Lorsque le stock d'un produit descend sous `stockMinimum`, une alerte de réapprovisionnement est générée
- Le stock ne peut jamais dépasser `stockMaximum` lors d'une entrée
- Une commande fournisseur ne peut être annulée que si son statut est **en attente**

7.3. Partie 1 : Modélisation

Réaliser le MCD selon Merise puis déduire le MLD complet.

7.4. Partie 2 : Base de données MySQL

Création (LDD)

Créer la base `entrepot_bd` et toutes ses tables avec les contraintes appropriées.

Alimentation (LMD)

Insérer au minimum : 4 catégories, 25 produits, 6 fournisseurs, 3 points de vente, 15 commandes fournisseurs, 20 mouvements de stock.

Requêtes de consultation

1. Produits dont le stock actuel est en dessous du stock minimum (alerte réapprovisionnement)
2. Valeur totale du stock par catégorie (prix d'achat $\times$ stock actuel)
3. Fournisseur le plus fiable (délai de livraison réel moyen le plus court)
4. Produits les plus sortis du stock sur les 30 derniers jours
5. Commandes fournisseurs en retard (date de livraison prévue dépassée, statut non livré)
6. Mouvements de stock d'un produit donné sur les 3 derniers mois

7.5. Partie 3 : Gestion des droits (LCD)

Créer deux utilisateurs :

- **magasinier** : SELECT sur toutes les tables ; INSERT sur **MouvementStock** uniquement
- **responsable_achat** : tous les droits sur **CommandeFourn**, **LigneCommande**, **Fournisseur** ; SELECT sur le reste

7.6. Partie 4 : Application cliente

Implémenter les fonctionnalités suivantes :

1. **Gestion des produits** : ajouter, modifier, consulter la fiche d'un produit avec son stock actuel
2. **Enregistrer une entrée de stock** : saisir une réception de commande fournisseur, mettre à jour le stock
3. **Enregistrer une sortie de stock** : vérifier la disponibilité, mettre à jour le stock, alerter si seuil minimum atteint
4. **Passer une commande fournisseur** : créer une commande avec ses lignes, calculer le montant total
5. **Historique des mouvements** : afficher les entrées/sorties d'un produit avec filtrage par période
6. **Tableau de bord** : alertes de réapprovisionnement, valeur totale du stock, commandes en retard

Bonne chance !

8. Sujet 8 : Gestion d'un Club Sportif

8.1. Contexte

Un club omnisports (football, basketball, volleyball) souhaite digitaliser la gestion de ses membres, de ses équipes, des entraînements, des compétitions et des cotisations annuelles. Le secrétaire du club doit pouvoir suivre les présences aux entraînements et gérer les licences des joueurs.

8.2. Entités et règles de gestion

- **Sport** : identifiant, nom, description
- **Membre** : numero de licence, nom, prenom, date de naissance, telephone, email, date d'adhesion
- **Equipe** : code, nom, sport associée, categorie (senior/junior/cadet), entraîneur
- **Appartenance** : membre, equipe, date d'entree, date de sortie (nulle si toujours actif), poste
- **Entraînement** : equipe, date, heure debut, duree, lieu, theme
- **Presence** : entraînement, membre, present (oui/non), motif d'absence
- **Compétition** : nom, sport, date, lieu, type (championnat/coupe/amical)
- **Participation** : equipe, competition, resultat (score ou classement)
- **Cotisation** : membre, saison (ex : 2025-2026), montant, date de paiement, statut (payee/impayee)

Règles de gestion :

- Un membre ne peut appartenir qu'à une seule équipe par sport à la fois
- Un membre avec cotisation impayée ne peut pas participer aux compétitions officielles
- Le taux d'assiduité d'un membre est calculé : présences / total entraînements de son équipe
- Un membre doit avoir au moins 16 ans pour rejoindre la catégorie senior

8.3. Partie 1 : Modélisation

Réaliser le MCD selon Merise puis déduire le MLD complet.

8.4. Partie 2 : Base de données MySQL

Création (LDD)

Créer la base de données `club_sport` et toutes ses tables avec les contraintes appropriées.

Alimentation (LMD)

Insérer au minimum : 3 sports, 6 équipes, 40 membres, 20 entraînements, 25 présences, 5 compétitions, 20 cotisations.

Requêtes de consultation

1. Taux d'assiduité par membre pour la saison en cours (GROUP BY + calcul)
2. Membres avec cotisation impayée pour la saison en cours
3. Meilleur buteur/marqueur par équipe (selon le sport)
4. Équipes classées par nombre de victoires en compétition
5. Membres présents à tous les entraînements du mois (EXISTS)
6. Membres n'ayant jamais participé à une compétition officielle

8.5. Partie 3 : Gestion des droits (LCD)

Créer deux utilisateurs :

- `entraîneur_role` : SELECT sur toutes les tables ; INSERT, UPDATE sur `Entrainement` et `Presence`
- `secretaire_club` : SELECT, INSERT, UPDATE sur `Membre`, `Cotisation`, `Appartenance` ; SELECT sur le reste

8.6. Partie 4 : Application cliente

Implémenter les fonctionnalités suivantes :

1. **Gestion des membres** : inscrire un membre, affecter à une équipe, consulter sa fiche (assiduité, cotisations, compétitions)
2. **Enregistrer un entraînement** : créer la séance et saisir les présences/absences des membres de l'équipe
3. **Gestion des cotisations** : enregistrer un paiement de cotisation, afficher les membres en retard
4. **Gestion des compétitions** : créer une compétition, inscrire des équipes, saisir les résultats
5. **Feuille de présence** : afficher la liste de présence d'un entraînement et calculer le taux d'assiduité
6. **Tableau de bord** : membres actifs, cotisations impayées, prochaines compétitions, meilleurs assidus

Bonne chance !

9. Sujet 9 : Gestion d'une Résidence Universitaire

9.1. Contexte

La résidence universitaire d'un campus souhaite se doter d'un système d'information pour assurer le suivi de ses bâtiments, de ses chambres, de l'attribution des chambres aux étudiants, des paiements de loyers mensuels et des incidents techniques signalés. Le responsable doit pouvoir détecter les impayés et gérer les demandes de maintenance.

9.2. Entités et règles de gestion

- **Batiment** : code, nom, adresse, nombre d'étages, année de construction
- **Chambre** : numero, batiment, type (**simple/double**), superficie, loyer mensuel, état (**disponible/occupee/en travaux**)
- **Etudiant** : numero etudiant, nom, prenom, filiere, niveau (**L1/L2/L3/M1/M2**), telephone, email
- **Attribution** : etudiant, chambre, date d'entrée, date de sortie prévue, caution versée
- **PaiementLoyer** : attribution, mois, année, montant, date de paiement effectif
- **Incident** : chambre, date de signalement, description, priorité (**faible/moyenne/urgente**), statut (**ouvert/en cours/resolu**), date de résolution

Règles de gestion :

- Une chambre **en travaux** ne peut pas être attribuée
- Un étudiant ne peut avoir qu'une seule attribution active à la fois
- La caution est égale à un mois de loyer et est restituée à la sortie si aucun incident n'est imputable à l'étudiant
- Un loyer est considéré en retard s'il n'est pas payé avant le 5 du mois concerné

9.3. Partie 1 : Modélisation

Réaliser le MCD selon Merise puis déduire le MLD complet.

9.4. Partie 2 : Base de données MySQL

Création (LDD)

Créer la base `residence_univ` et toutes ses tables avec les contraintes appropriées.

Alimentation (LMD)

Insérer au minimum : 3 bâtiments, 30 chambres, 25 étudiants, 20 attributions actives, 50 paiements de loyer, 10 incidents.

Requêtes de consultation

1. Chambres disponibles par bâtiment avec leur loyer
2. Étudiants n'ayant pas payé le loyer du mois en cours (sous-requête avec NOT EXISTS)
3. Taux d'occupation par bâtiment (nb chambres occupées / nb total $\times$ 100)
4. Incidents urgents non résolus avec le bâtiment et la chambre concernés
5. Revenus locatifs encaissés par mois sur les 6 derniers mois
6. Étudiants dont la date de sortie prévue est dans moins de 30 jours

9.5. Partie 3 : Gestion des droits (LCD)

Créer deux utilisateurs :

- `gestionnaire_res` : SELECT, INSERT, UPDATE sur toutes les tables; pas de DELETE
- `technicien` : SELECT sur `Chambre`, `Batiment`; SELECT, UPDATE sur `Incident` uniquement

9.6. Partie 4 : Application cliente

Implémenter les fonctionnalités suivantes :

1. **Gestion des étudiants** : inscrire, modifier, afficher la fiche d'un étudiant (attribution actuelle, historique des paiements)
2. **Attribuer une chambre** : rechercher les chambres disponibles, vérifier les règles, enregistrer l'attribution et la caution
3. **Enregistrer un paiement de loyer** : saisir mois/année/montant ; alerter si le paiement est en retard
4. **Déclarer un incident** : saisir la chambre, la description et la priorité ; mettre à jour le statut
5. **Gestion des départs** : enregistrer la sortie d'un étudiant, vérifier les impayés, restituer ou retenir la caution
6. **Tableau de bord** : taux d'occupation global, loyers impayés du mois, incidents urgents non résolus

Bonne chance !

10. Sujet 10 : Gestion d'une Entreprise et de sa Paie

10.1. Contexte

Une PME souhaite moderniser et structurer le suivi de ses ressources humaines : employés, départements, contrats de travail, présences/absences, congés et calcul des salaires mensuels. Le directeur RH doit pouvoir générer les bulletins de paie et suivre les absences non justifiées.

10.2. Entités et règles de gestion

- **Département** : code, nom, chef de département, budget annuel
- **Poste** : code, intitulé, salaire de base, niveau de responsabilité
- **Employé** : matricule, nom, prénom, date de naissance, date d'embauche, email, téléphone, département, poste
- **Contrat** : employé, type (CDI/CDD/Stage), date de début, date de fin, salaire brut
- **Presence** : employé, date, heure d'arrivée, heure de départ, statut (present/absent/conge)
- **Conge** : employé, date de début, date de fin, type (annuel/maladie/maternite), statut (demande/approuve/refuse)
- **Bulletin** : employé, mois, année, salaire brut, nb jours travaillés, nb absences injustifiées, retenues, salaire net

Règles de gestion :

- Chaque absence injustifiée entraîne une retenue de **1/22 du salaire brut mensuel**
- Un employé ne peut pas avoir deux contrats actifs simultanément
- Le solde de congés annuels est de 30 jours par an ; un congé refusé ou annulé restitue les jours
- Un congé ne peut être approuvé que si l'employé a un contrat actif

10.3. Partie 1 : Modélisation

Réaliser le MCD selon Merise puis déduire le MLD complet.

10.4. Partie 2 : Base de données MySQL

Création (LDD)

Créer la base de données `rh_entreprise` et toutes ses tables avec les contraintes appropriées.

Alimentation (LMD)

Insérer au minimum : 4 départements, 8 postes, 20 employés, 20 contrats, 30 présences, 10 congés, 20 bulletins.

Requêtes de consultation

1. Masse salariale mensuelle par département (GROUP BY + SUM)
2. Employés ayant plus de 3 absences injustifiées ce mois (HAVING)
3. Employés dont le contrat CDD expire dans moins de 30 jours
4. Solde de congés restants par employé pour l'année en cours
5. Employé le mieux payé par département (sous-requête corrélée)
6. Taux d'absentéisme par département sur les 3 derniers mois

10.5. Partie 3 : Gestion des droits (LCD)

Créer deux utilisateurs :

- `rh_manager` : tous les droits sur toutes les tables de la base
- `employe_role` : SELECT uniquement sur `Conge` et `Bulletin` (ses propres données via une vue si possible)

10.6. Partie 4 : Application cliente

Implémenter les fonctionnalités suivantes :

1. **Gestion des employés** : ajouter, modifier, afficher la fiche complète d'un employé (contrat, absences, congés)
2. **Enregistrer les présences** : saisir les présences/absences d'une journée pour un département entier
3. **Gérer les congés** : soumettre une demande, approuver ou refuser, déduire du solde disponible
4. **Générer un bulletin de paie** : calculer automatiquement les retenues pour absences injustifiées et le salaire net
5. **Suivi des contrats** : lister les CDD expirant bientôt, afficher les employés sans contrat actif
6. **Tableau de bord RH** : masse salariale du mois, taux d'absentéisme, congés en attente d'approbation

Bonne chance !